

Dispositif de détection des événements de sécurité

Compte rendu destiné au formateur — Olivier HIMBLOT

Formation AIS — Travaux des 7 et 8 octobre 2026 — Document établi le 9 octobre 2026

Objectif évalué

Mettre en œuvre un dispositif de détection d'événements de sécurité et vérifier son fonctionnement.

Critères : configurer les sources de détection ; générer et retrouver des événements ; interpréter les alertes ; identifier les limites du dispositif.

Preuve de travail : dispositif fonctionnel et preuves de détection comprenant l'analyse d'activités réseau avec Suricata et l'exploitation des informations disponibles dans Wazuh.

Résultat documenté

Suricata est actif et produit des alertes sur des marqueurs HTTP contrôlés : quatre signatures sur un serveur temporaire, puis une règle locale ciblant File Browser sur 8080. Wazuh reçoit les journaux Docker de File Browser via un agent installé sur la VM ; trois messages `/api/renew 401` ont été retrouvés dans les archives du manager.

La détection réseau et la collecte applicative sont documentées. Une alerte Wazuh spécifique à ces messages et la centralisation des événements Suricata dans Wazuh ne sont pas démontrées.

Périmètre des preuves

Ce compte rendu synthétise les captures et sorties déjà conservées dans le mémo. Il ne constitue pas un nouveau contrôle en direct. L'activité Suricata est individuelle ; la plateforme Wazuh relève du travail collectif, complété par les essais documentés de collecte File Browser. Les adresses privées servent à corréler les preuves du laboratoire.

1. Sources configurées et architecture

Source réseau : Suricata 8.0.3 installé sur l'hôte et configuré sur virbr0 (192.168.122.1/24). La VM File Browser utilise 192.168.122.229 ; Docker publie le service HTTP sur 8080. Un serveur HTTP temporaire sur 18080 a servi aux premiers essais.

Chemin réseau : hôte → bridge virbr0 observé par Suricata → VM File Browser. Journal lu : /var/log/suricata/eve.json. Types observés : alert, http, dns, tls, flow, fileinfo et ssh.

Le premier démarrage échouait car eth0 était absente. Après sélection de virbr0, le test de configuration a accepté quatre signatures sans erreur et le service est devenu actif le 7 octobre à 10:18:27. Le 8 octobre, sept signatures dans deux fichiers ont été acceptées, code 0.

Source applicative : File Browser → stdout/stderr → fichier Docker json-file → agent Wazuh sur la VM → manager Wazuh 192.168.122.37:1514/TCP. Agent 001, vm-filebrowser, version 4.14.8, Ubuntu 26.04.1 LTS. Le manager est également en 4.14.8 (révision rc2 documentée).

Le collecteur utilise un bloc localfile avec log_format=json et le chemin réel obtenu par docker inspect --format {{.LogPath}}. Son test a retourné 0 ; le fichier Docker est annoncé comme analysé. L'agent se trouve sur la VM, pas dans le conteneur BusyBox-musl.

```
vnet1 UNKNOWN fe80::fc54:ff:fee9:d64c/64
oliv@ubuntu-oliv:~$ sudoedit /etc/suricata/suricata.yaml
oliv@ubuntu-oliv:~$ sudo suricata -T -v -c /etc/suricata/suricata.yaml
echo "Code de retour : $?"
Notice: suricata: This is Suricata version 8.0.3 RELEASE running in SYSTEM mode
Info: cpu: CPUs/cores online: 20
Info: suricata: Running suricata under test mode
Info: suricata: Setting engine mode to IDS mode by default
Info: exception-policy: master exception-policy set to: auto
Info: suricata: Preparing unexpected signal handling
Info: logopenfile: fast output device (regular) initialized: fast.log
Info: logopenfile: eve-log output device (regular) initialized: eve.json
Info: logopenfile: stats output device (regular) initialized: stats.log
Info: detect: 1 rule files processed. 4 rules successfully loaded, 0 rules failed, 0 rules skipped
Info: threshold-config: Threshold config parsed: 0 rule(s) found
Info: detect: 4 signatures processed. 0 are IP-only rules, 0 are inspecting packet payload, 4 inspect application layer, 0 are decoder event only
Notice: mpn-hs: Rule group caching - loaded: 2 newly cached: 0 total cacheable: 2
Notice: suricata: Configuration provided was successfully loaded. Exiting.
Code de retour : 0
oliv@ubuntu-oliv:~$ sudo systemctl reset-failed suricata
sudo systemctl start suricata
sudo systemctl status suricata --no-pager
● suricata.service - Suricata IDS/IDP daemon
   Loaded: loaded (/usr/lib/systemd/system/suricata.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-10-07 10:18:27 CEST; 13ms ago
  Invocation: 04d9354630b74f7b942f138a38143830
     Docs: man:suricata(8)
           man:suricatasc(8)
           https://suricata.io/documentation/
    Main PID: 129681 (Suricata-Main)
      Tasks: 26 (limit: 37430)
    Memory: 84.8M (peak: 84.8M)
       CPU: 151ms
    CGroup: /system.slice/suricata.service
            └─129681 /usr/bin/suricata -D --af-packet -c /etc/suricata/suricata.yaml --pidfile /var/run/suricata/suricata.pid

oct. 07 10:18:27 ubuntu-oliv systemd[1]: Starting suricata.service - Suricata IDS/IDP daemon...
oct. 07 10:18:27 ubuntu-oliv suricata[129680]: i: suricata: This is Suricata version 8.0.3 RELEASE running in SYSTEM mode
oct. 07 10:18:27 ubuntu-oliv systemd[1]: Started suricata.service - Suricata IDS/IDP daemon.
oliv@ubuntu-oliv:~$
```

P1 — Configuration Suricata acceptée et service actif après correction de l'interface.

2. Générer et retrouver les alertes Suricata

Le 7 octobre, quatre requêtes HTTP contrôlées ont envoyé des marqueurs de traversée ou de script vers le serveur temporaire de la VM:18080. Le serveur répond 404 ; les alertes apparaissent dans eve.json.

SID 50500003 : %2e%2e dans l'URI brute — 10:30:38.887961 +02:00. SID 50500004 : ..%2f — 10:30:38.892375. SID 50500006 : %252e%252e — 10:30:38.898569. SID 50100004 : ouverture de balise script dans l'URI normalisée — 10:30:38.904860.

```
10:30:28.627472 IP 192.168.122.229.10800 > 192.168.122.1.57374: Flags [P.], seq 1861646, ack 110, win 128, options [nop,nop,TS val 1202132165 ecr 410504930], length 460
10:30:28.627474 IP 192.168.122.1.57374 > 192.168.122.229.10800: Flags [.], ack 646, win 63, options [nop,nop,TS val 410504930 ecr 1202132165], length 0
10:30:28.627479 IP 192.168.122.229.10800 > 192.168.122.1.57374: Flags [F.], seq 646, ack 110, win 128, options [nop,nop,TS val 1202132165 ecr 410504930], length 0
10:30:28.627565 IP 192.168.122.1.57374 > 192.168.122.229.10800: Flags [F.], seq 110, ack 647, win 63, options [nop,nop,TS val 410504930 ecr 1202132165], length 0
10:30:28.627577 IP 192.168.122.229.10800 > 192.168.122.1.57374: Flags [.], ack 111, win 128, options [nop,nop,TS val 1202132165 ecr 410504930], length 0
10:30:38.886950 IP 192.168.122.1.52274 > 192.168.122.229.10800: Flags [S.], seq 572894967, win 64240, options [mss 1460,sackOK,TS val 4079290589 ecr 0,nop,wscale 10], length 0
10:30:38.887150 IP 192.168.122.229.10800 > 192.168.122.1.52274: Flags [S.], seq 218094380, ack 572894969, win 65160, options [mss 1460,sackOK,TS val 2893787691 ecr 4079290589,nop,wscale 9], length 0
10:30:38.887196 IP 192.168.122.1.52274 > 192.168.122.229.10800: Flags [.], ack 1, win 63, options [nop,nop,TS val 4079290589 ecr 2893787691], length 0
10:30:38.887293 IP 192.168.122.1.52274 > 192.168.122.229.10800: Flags [P.], seq 1:107, ack 1, win 63, options [nop,nop,TS val 4079290589 ecr 2893787691], length 106
10:30:38.887439 IP 192.168.122.229.10800 > 192.168.122.1.52274: Flags [.], ack 107, win 128, options [nop,nop,TS val 2893787691 ecr 4079290589], length 0
10:30:38.887932 IP 192.168.122.229.10800 > 192.168.122.1.52274: Flags [P.], seq 1:186, ack 107, win 128, options [nop,nop,TS val 2893787692 ecr 4079290589], length 185
10:30:38.887961 IP 192.168.122.1.52274 > 192.168.122.229.10800: Flags [.], ack 186, win 63, options [nop,nop,TS val 4079290589 ecr 2893787692], length 0
10:30:38.887964 IP 192.168.122.229.10800 > 192.168.122.1.52274: Flags [FP.], seq 186:646, ack 107, win 128, options [nop,nop,TS val 2893787692 ecr 4079290589], length 460
10:30:38.888028 IP 192.168.122.1.52274 > 192.168.122.229.10800: Flags [F.], seq 107, ack 647, win 63, options [nop,nop,TS val 4079290590 ecr 2893787692], length 0
10:30:38.888043 IP 192.168.122.229.10800 > 192.168.122.1.52274: Flags [.], ack 188, win 128, options [nop,nop,TS val 2893787692 ecr 4079290590], length 0
10:30:38.891023 IP 192.168.122.1.52282 > 192.168.122.229.10800: Flags [S.], seq 1737873365, win 64240, options [mss 1460,sackOK,TS val 3744925180 ecr 0,nop,wscale 10], length 0
10:30:38.891873 IP 192.168.122.229.10800 > 192.168.122.1.52282: Flags [S.], seq 888024249, ack 1737873365, win 65160, options [mss 1460,sackOK,TS val 3428643430 ecr 3744925180,nop,wscale 9], length 0
10:30:38.891888 IP 192.168.122.1.52282 > 192.168.122.229.10800: Flags [.], ack 1, win 63, options [nop,nop,TS val 3744925180 ecr 3428643430], length 0
10:30:38.891904 IP 192.168.122.1.52282 > 192.168.122.229.10800: Flags [P.], seq 1:186, ack 1, win 63, options [nop,nop,TS val 3744925180 ecr 3428643430], length 185
10:30:38.891916 IP 192.168.122.229.10800 > 192.168.122.1.52282: Flags [.], ack 186, win 128, options [nop,nop,TS val 3428643430 ecr 3744925180], length 0
10:30:38.892302 IP 192.168.122.229.10800 > 192.168.122.1.52282: Flags [P.], seq 1:186, ack 186, win 128, options [nop,nop,TS val 3428643430 ecr 3744925180], length 185
10:30:38.892375 IP 192.168.122.1.52282 > 192.168.122.229.10800: Flags [.], ack 186, win 63, options [nop,nop,TS val 3744925181 ecr 3428643430], length 0
10:30:38.892387 IP 192.168.122.229.10800 > 192.168.122.1.52282: Flags [FP.], seq 186:646, ack 186, win 128, options [nop,nop,TS val 3428643430 ecr 3744925180], length 460
10:30:38.892456 IP 192.168.122.1.52282 > 192.168.122.229.10800: Flags [F.], seq 186, ack 647, win 63, options [nop,nop,TS val 3744925181 ecr 3428643430], length 0
10:30:38.892477 IP 192.168.122.229.10800 > 192.168.122.1.52282: Flags [.], ack 187, win 128, options [nop,nop,TS val 3428643430 ecr 3744925181], length 0
10:30:38.897815 IP 192.168.122.1.52296 > 192.168.122.229.10800: Flags [S.], seq 262927729, win 64240, options [mss 1460,sackOK,TS val 3942364230 ecr 0,nop,wscale 10], length 0
10:30:38.897887 IP 192.168.122.229.10800 > 192.168.122.1.52296: Flags [S.], seq 1977365172, ack 262927730, win 65160, options [mss 1460,sackOK,TS val 176382187 ecr 3942364230,nop,wscale 9], length 0
10:30:38.897900 IP 192.168.122.1.52296 > 192.168.122.229.10800: Flags [.], ack 1, win 63, options [nop,nop,TS val 3942364230 ecr 176382187], length 0
```

```
oliv@ubuntu-oliv:~$ sudo tail -n 0 -F /var/log/suricata/eve.json | jq --unbuffered -c '
select(event_type == "alert") |
select(alert.signature_id == 50500003 or
.alert.signature_id == 50500004 or
.alert.signature_id == 50500006 or
.alert.signature_id == 50100004) |
(timestamp,flow_id,src_ip,src_port,dest_ip,dest_port,
sid:alert.signature_id,signature:alert.signature,action:alert.action)'
```

```
oliv@ubuntu-oliv:~$ sudo tail -n 0 -F /var/log/suricata/eve.json | jq --unbuffered -c '
select(event_type == "alert") |
select(alert.signature_id == 50500003 or
.alert.signature_id == 50500004 or
.alert.signature_id == 50500006 or
.alert.signature_id == 50100004) |
(timestamp,flow_id,src_ip,src_port,dest_ip,dest_port,
sid:alert.signature_id,signature:alert.signature,action:alert.action)'
```

```
{
  "timestamp": "2020-10-07T10:30:38.887961+0200", "flow_id": "1838670500379292", "src_ip": "192.168.122.1", "src_port": "52274", "dest_ip": "192.168.122.229", "dest_port": "10800", "sid": "50500003", "signature": "Possible Path Traversal attack, percent-encoded slash traversal", "action": "allowed"
}, {
  "timestamp": "2020-10-07T10:30:38.892375+0200", "flow_id": "1838670500379292", "src_ip": "192.168.122.1", "src_port": "52282", "dest_ip": "192.168.122.229", "dest_port": "10800", "sid": "50500004", "signature": "Possible Path Traversal attack, percent-encoded double-encoded double-encoded", "action": "allowed"
}, {
  "timestamp": "2020-10-07T10:30:38.898569+0200", "flow_id": "19133326596272334", "src_ip": "192.168.122.1", "src_port": "52386", "dest_ip": "192.168.122.229", "dest_port": "10800", "sid": "50100004", "signature": "Possible XSS attack, script tag in decoded URI", "action": "allowed"
}
```

P2 — Trafic TCP sur 18080 et quatre SID corrélés aux essais contrôlés.

Pour retrouver une détection, filtrer EVE par event_type=alert, SID, destination et intervalle ; conserver ensuite l'objet JSON complet. Lire signature, timestamp, in_iface, src_ip, dest_ip, ports, flow_id et contexte HTTP. Le flow_id relie les événements d'un même flux.

Interprétation : les motifs sont détectés malgré la réponse 404. action:allowed indique que le trafic n'a pas été bloqué. Ces essais ne démontrent ni exploitation réussie, ni lecture de fichier sensible, ni exécution XSS. Ils ne valident pas à eux seuls le service sur 8080.

3. Détection adaptée au service File Browser

Le 8 octobre, une règle locale cible HTTP vers 192.168.122.229:8080 : SID 1008001, révision 1, motif %2e%2e dans http.uri.raw, flux établi vers le serveur. Elle adapte la condition de la signature générique au service observé.

```
oliv@ubuntu-oliv:~$ sudo suricata -T -v -c /etc/suricata/suricata.yaml
Code de retour : 5
Notice: suricata: This is Suricata version 8.0.3 RELEASE running in SYSTEM mode
Info: cpu: CPUs/cores online: 20
Info: suricata: Running suricata under test mode
Info: suricata: Setting engine mode to IDS mode by default
Info: exception-policy: master exception-policy set to: auto
Info: suricata: Preparing unexpected signal handling
Info: logopenfile: Fast output device (regular) initialized: fast.log
Info: logopenfile: eve-log output device (regular) initialized: eve.json
Info: logopenfile: stats output device (regular) initialized: stats.log
Info: detect: 2 rule files processed: 7 rules successfully loaded, 0 rules failed, 0 rules skipped
Info: threshold-config: Threshold config parsed: 0 rule(s) found
Info: detect: 7 signatures processed, 0 are IP-only rules, 0 are inspecting packet payload, 7 inspect application layer, 0 are decoder event only
Notice: mp-mhs: Rule group caching - loaded: 0 newly cached: 5 total cacheable: 5
Notice: suricata: Configuration provided was successfully loaded. Exiting.
Code de retour : 0
oliv@ubuntu-oliv:~$ sudo systemctl restart suricata
sudo systemctl status suricata --no-pager
● suricata.service - Suricata IDS/IDP daemon
   Loaded: loaded (/usr/lib/systemd/system/suricata.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-10-08 11:52:19 CEST; 11ms ago
     Invocation: d93c933bcc1f4f409eb2142989cc9125
       Docs: man:suricata(8)
            man:suricatasc(8)
            https://suricata.io/documentation/
    Main PID: 179269 (Suricata-Main)
      Tasks: 26 (limit: 37430)
    Memory: 82.0M (peak: 83.5M)
       CPU: 139ms
    CGroup: /system.slice/suricata.service
            └─179269 /usr/bin/suricata -D --af-packet -c /etc/suricata/suricata.yaml --pidfile /var/run/suricata/suricata.pid

oct. 08 11:52:19 ubuntu-oliv systemd[1]: Starting suricata.service - Suricata IDS/IDP daemon...
oct. 08 11:52:19 ubuntu-oliv suricata[179266]: i: suricata: This is Suricata version 8.0.3 RELEASE running in SYSTEM mode
oct. 08 11:52:19 ubuntu-oliv systemd[1]: Started suricata.service - Suricata IDS/IDP daemon.
oliv@ubuntu-oliv:~$ FB_BASE="http://192.168.122.229:8080"
# Motif inerte dans un paramètre, sans chemin vers fichier sensible.
curl --path-as-is --max-time 5 -o /dev/null "$FB_BASE"/ats_probe=%2e%2e"
# Série normale à faible volume, sans query.
for essai in 1 2 3 4 5; do
  curl --max-time 5 -o /dev/null -s "$FB_BASE/"
done
sleep 1
done
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100  5738    0  5738    0    3.95M    0      0      0      0      0      0      0
```

P3 — Sept signatures acceptées, service actif et alerte locale 1008001 sur File Browser/8080.

Événement : 2026-10-08T11:53:20.484380+0200 ; source 192.168.122.1:55214 ; destination 192.168.122.229:8080 ; flow_id 105860300332322 ; GET /?ais_probe=%2e%2e ; réponse 200 ; action allowed.

Analyse : le paramètre est un marqueur inerte. L'alerte confirme la détection sur 8080 ; HTTP 200 ne prouve aucune exploitation. Le contrôle négatif de cette règle reste à documenter.

Deux autres règles sont chargées : 1008002 vise cinq GET / en dix secondes ; 1008003 vise les réponses 401. Aucune alerte correspondante n'est visible dans cette capture : leur déclenchement reste à valider. Cela ne prouve pas leur absence dans le journal complet.

4. Activités normales et limites du capteur réseau

```
{ "timestamp": "2020-10-07T10:59:53.030Z", "event_type": "fileinfo", "flow_id": "890875425894290", "la_iface": "virbr0", "src_ip": "192.168.122.229", "src_port": "8080", "dest_ip": "192.168.122.1", "dest_port": "42426", "proto": "TCP", "app_proto": "http", "http": { "hostname": "192.168.122.229", "http_port": "8080", "url": "/", "http_user_agent": "curl/8.10.0", "http_content_type": "text/html", "http_method": "GET", "protocol": "HTTP/1.1", "status": "200", "length": "5762", "ssh": null, "flow": null, "alert": null } }, { "timestamp": "2020-10-07T10:59:54.051092+0200", "event_type": "http", "flow_id": "836422646266094", "la_iface": "virbr0", "src_ip": "192.168.122.1", "src_port": "42424", "dest_ip": "192.168.122.229", "dest_port": "8080", "proto": "TCP", "app_proto": null, "http": { "hostname": "192.168.122.229", "http_port": "8080", "url": "/", "http_user_agent": "curl/8.10.0", "http_content_type": "text/html", "http_method": "GET", "protocol": "HTTP/1.1", "status": "200", "length": "5762", "ssh": null, "flow": null, "alert": null } }, { "timestamp": "2020-10-07T10:59:54.051216+0200", "event_type": "fileinfo", "flow_id": "836422646266094", "la_iface": "virbr0", "src_ip": "192.168.122.229", "src_port": "8080", "dest_ip": "192.168.122.1", "dest_port": "42424", "proto": "TCP", "app_proto": "http", "http": { "hostname": "192.168.122.229", "http_port": "8080", "url": "/", "http_user_agent": "curl/8.10.0", "http_content_type": "text/html", "http_method": "GET", "protocol": "HTTP/1.1", "status": "200", "length": "5762", "ssh": null, "flow": null, "alert": null } }, { "timestamp": "2020-10-07T10:59:55.063506+0200", "event_type": "http", "flow_id": "891031043541603", "la_iface": "virbr0", "src_ip": "192.168.122.1", "src_port": "42436", "dest_ip": "192.168.122.229", "dest_port": "8080", "proto": "TCP", "app_proto": null, "http": { "hostname": "192.168.122.229", "http_port": "8080", "url": "/", "http_user_agent": "curl/8.10.0", "http_content_type": "text/html", "http_method": "GET", "protocol": "HTTP/1.1", "status": "200", "length": "5762", "ssh": null, "flow": null, "alert": null } }, { "timestamp": "2020-10-07T10:59:55.063761+0200", "event_type": "fileinfo", "flow_id": "891031043541603", "la_iface": "virbr0", "src_ip": "192.168.122.229", "src_port": "8080", "dest_ip": "192.168.122.1", "dest_port": "42436", "proto": "TCP", "app_proto": "http", "http": { "hostname": "192.168.122.229", "http_port": "8080", "url": "/", "http_user_agent": "curl/8.10.0", "http_content_type": "text/html", "http_method": "GET", "protocol": "HTTP/1.1", "status": "200", "length": "5762", "ssh": null, "flow": null, "alert": null } }
```

P4 — Accès HTTP successifs à File Browser retrouvés dans EVE.

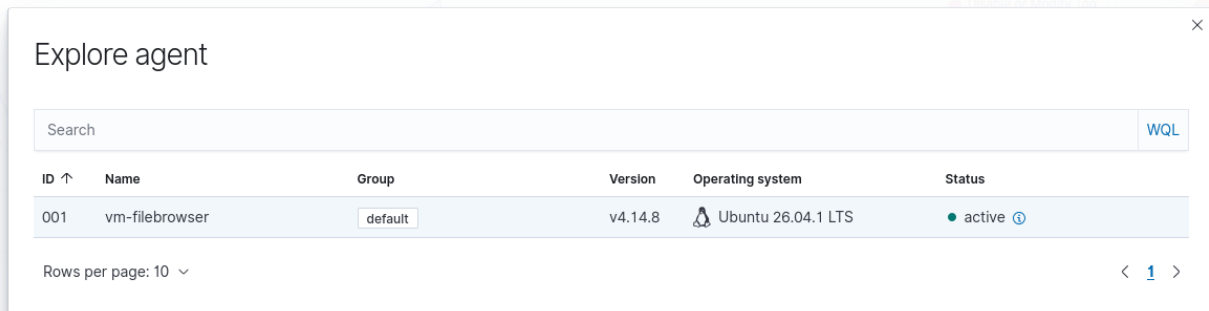
Les observations du 7 octobre montrent le chargement de la page et de ses ressources, puis cinq GET retournant 200 entre 10:59:51 et 10:59:55. Les bilans sélectionnés affichent alerted:false. Une même activité génère plusieurs objets HTTP, fileinfo et flow ; le nombre de lignes ne mesure pas le nombre d'actions humaines.

Des flux DNS et TLS sortants sont également visibles. Ils renseignent destinations et métadonnées, sans établir une exfiltration ou une commande malveillante. Un 401 sur /api/renew n'est pas une preuve de mauvais mot de passe.

Couverture : virbr0 prouve les trajets testés, sans couvrir automatiquement le loopback VM, les échanges internes Docker ou tous les autres réseaux. HTTPS et SSH peuvent laisser voir des métadonnées tout en masquant le contenu.

Les règles reconnaissent des conditions précises ; elles ne prouvent ni identité, ni intention, ni vulnérabilité exploitable. Aucun taux de faux positifs ou volume quotidien n'a été mesuré. Le mode IDS ne remplace pas le filtrage réseau. L'arrêt du serveur temporaire reste à confirmer.

5. Wazuh : agent actif et collecte applicative



The screenshot shows the 'Explore agent' window in Wazuh. It features a search bar at the top with the text 'Search' and a 'WQL' button. Below the search bar is a table with columns: ID, Name, Group, Version, Operating system, and Status. The table contains one row for agent 001, named 'vm-filebrowser', with group 'default', version 'v4.14.8', and operating system 'Ubuntu 26.04.1 LTS'. The status is 'active' with a green dot and an information icon. At the bottom, there is a 'Rows per page: 10' dropdown and pagination controls showing page 1 of 1.

ID ↑	Name	Group	Version	Operating system	Status
001	vm-filebrowser	default	v4.14.8	Ubuntu 26.04.1 LTS	● active ⓘ

P5 — Agent 001 vm-filebrowser actif, version 4.14.8, Ubuntu 26.04.1 LTS.

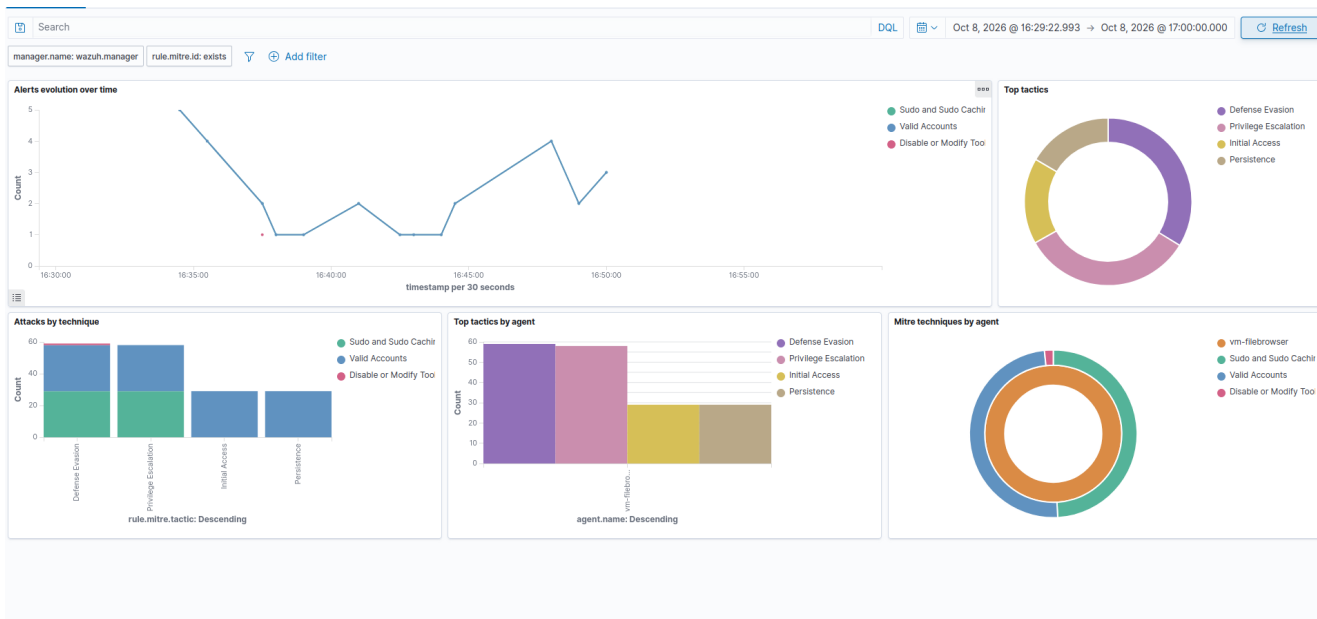
Le statut actif confirme la connexion de l'agent ; la réception applicative repose sur les événements retrouvés dans archives.json du manager. Trois messages /api/renew 401 sont documentés le 8 octobre à 14:49:05.845, 14:49:43.848 et 14:50:07.851 UTC (16:49-16:50 CEST).

Champs conservés du dernier message : agent.id=001 ; agent.name=vm-filebrowser ; agent.ip=192.168.122.229 ; manager.name=wazuh.manager ; decoder.name=json ; data.stream=stdout ; data.time=2026-10-08T14:50:07.29758532Z.

data.log : 2026/10/08 14:50:07 /api/renew: 401 192.168.122.1 <nil>. Ces champs proviennent de l'extrait texte fourni, pas d'un objet brut complet annexé à ce PDF.

Analyse : collecte de bout en bout démontrée et décodage JSON obtenu. Le statut 401 représente un refus de renouvellement de session. Le chemin HTTP, le statut et l'adresse restent dans data.log ; aucun décodeur applicatif dédié ni règle Wazuh spécifique n'est validé ici.

6. Exploitation des informations Wazuh et diagnostic



P6 — Vue MITRE Wazuh avec événements associés à vm-filebrowser.

La vue MITRE affiche des agrégations de règles associées à des techniques. Cette information oriente l'analyse ; elle ne prouve ni attaque ni compromission. Pour qualifier un événement, consulter son détail, l'agent, le journal source, l'heure, la règle et le contexte des opérations du laboratoire.

Diagnostic réalisé : les premières recherches ne trouvaient pas les logs applicatifs car `logall_json` était à `no` dans la configuration effective du manager. Le fichier source monté avait été changé à `yes`, mais la configuration persistante `/var/ossec/etc/ossec.conf` restait à `no`.

Après sauvegarde et correction de la configuration effective, redémarrage et nouvelle requête, les messages ont été retrouvés dans `archives.json`. La divergence des fichiers a été constatée ; son mécanisme exact n'a pas été établi.

Ne rien retrouver dans `alerts.json` ne démontre pas une absence de transmission : un événement reçu et archivé ne devient pas nécessairement une alerte. Les archives locales ne sont pas automatiquement visibles dans le dashboard. L'archivage complet était encore actif au dernier état observé ; sa réduction après diagnostic reste à vérifier.

7. Bilan par critère et suites nécessaires

Configurer les sources — Documenté : interface virbr0, signatures Suricata acceptées, agent Wazuh sur la VM et source Docker JSON configurée. La configuration complète et sa continuité après recréation restent à conserver et tester.

Générer et retrouver — Documenté : quatre alertes sur 18080, alerte locale 1008001 sur 8080, activités normales EVE et trois messages applicatifs reçus dans les archives Wazuh.

Interpréter — Documenté : corrélation par heures, adresses, ports et flux ; distinction motif/exploitation, observation/blocage, réception/alerte et classification MITRE/attaque prouvée.

Identifier les limites — Documenté : contenu chiffré, trajets hors capteur, bruit non mesuré, règles 1008002/1008003 non validées, absence d’alerte applicative Wazuh démontrée et absence d’ingestion EVE dans Wazuh prouvée.

À compléter

Exporter les objets EVE et Wazuh originaux sans secret ; réaliser les contrôles négatifs et les essais des deux règles restantes ; définir un décodeur/règle File Browser puis prouver l’alerte ; vérifier rotation des logs, recréation du conteneur et rétention. Le chemin Docker dépend de l’identité du conteneur. La politique SCA Ubuntu 22.04 a été ignorée sur Ubuntu 26.04 : couverture de conformité limitée.

Sources du compte rendu

[docs/securisation-avancee-infrastructures/it-7/installer-tester-regles-suricata.md](#)

[docs/securisation-avancee-infrastructures/it-7/bilan-dispositif-detection.md](#)

[docs/securisation-avancee-infrastructures/it-8/rechercher-adapter-regles-detection.md](#)

[docs/securisation-avancee-infrastructures/it-8/etudier-agent-wazuh-file-browser.md](#)

[docs/securisation-avancee-infrastructures/it-8/installer-wazuh-single-node.md](#)

[docs/securisation-avancee-infrastructures/it-8/relier-detections-mitre-attack.md](#)

Les captures annexées sont les pièces déjà présentes dans le mémo. Disponibilité durable, couverture exhaustive et réussite d’une attaque ne sont pas déduites de ces preuves datées.